

Capítulo 17

Classes: uma visão mais detalhada, parte 1

Exercícios de Autorrevisão — Resolução Didática

“No Capítulo 16 dimos o passo do procedural para o orientado a objetos. Agora aprofundamos: separação explícita entre interface (`.h`) e implementação (`.cpp`), construtores e destrutores, atribuição default de objetos, e o operador binário `::` em todo o seu vigor. Os exercícios de autorrevisão deste capítulo são curtos — apenas dois —, mas concentrados em pontos sutis que costumam confundir o iniciante.”

1 Exercício 17.1 — Operadores de acesso e visibilidade

Preencha os espaços em cada uma das sentenças (a)–(d).

(a) Operadores que acessam membros de classe

Resposta: operador **ponto** (`.`) em conjunto com o nome de um objeto ou referência; operador **seta** (`->`) em conjunto com um ponteiro para um objeto.

Há duas situações distintas em que precisamos acessar um membro de uma classe a partir de um objeto:

1. **Quando temos o objeto diretamente (ou uma referência a ele):** usa-se o operador ponto.

```
Conta c(100);           // c e um objeto
c.credit(50);           // acesso via "."
Conta &r = c;            // r e uma referencia
r.credit(20);           // tambem "." (referencia e como apelido)
```

2. **Quando temos um ponteiro para o objeto:** usa-se o operador seta.

```
Conta *p = &c;           // p e ponteiro
p->credit(50);            // acesso via "->"
// equivalente a (*p).credit(50)
```

Observação de Engenharia de Software

O operador `->` é uma abreviação didática e sintaticamente conveniente para `(*p)`. Os parênteses em `(*p).credit(50)` são indispensáveis em razão das precedências: sem eles, `*p.credit(50)` seria interpretado como `*(p.credit(50))`, que não tem sentido (`p` é ponteiro, não tem membros diretamente). Por isso o operador `->` foi criado: torna o

código legível e elimina o risco de erro de precêndencia.

(b) Membros visíveis somente dentro da classe

Resposta: `private`.

Os membros `private` formam a *implementação* oculta da classe. Apenas as funções-membro da própria classe e suas *amigas* (*friends*) podem acessá-los. Tipicamente, os dados-membro vão para a seção `private`, e qualquer função auxiliar que não deve ser exposta aos clientes também vai para lá.

(c) Membros acessíveis onde quer que o objeto esteja no escopo

Resposta: `public`.

Os membros `public` formam a *interface* da classe — aquilo que os clientes podem usar. Tipicamente, contem os construtores, o destrutor, e as funções-membro que oferecem serviços aos clientes.

Boa Prática de Programação

A regra de ouro do encapsulamento: *dados-membro em `private`, funções-membro que oferecem serviços em `public`*. Exceções existem (constantes públicas, classes de políticas), mas são excepcionais. Quando um dado-membro precisa ser exposto, faça-o através de *getters/setters* — como vimos no Capítulo 16.

(d) Forma de atribuir um objeto a outro da mesma classe

Resposta: a **atribuição de membros default** (*default memberwise assignment*), realizada pelo operador de atribuição `=`.

C++ gera, automaticamente, um operador de atribuição para cada classe que você define. Esse operador padrão copia, um a um, todos os dados-membro do objeto da direita para o objeto da esquerda:

```
Conta c1(100);  
Conta c2;  
c2 = c1;           // atribui c1 a c2: copia cada dado-membro
```

Erro Comum de Programação

A atribuição default funciona maravilhosamente para classes simples (cujos dados-membro são todos primitivos ou são tipos com semantica adequada de cópia). Mas, para classes que gerenciam recursos — ponteiros para memória dinâmica, descritores de arquivo, conexões de rede —, a atribuição default é *tipicamente perigosa*: dois objetos passam a apontar para o mesmo recurso, e o primeiro a ser destruído o libera, deixando o outro com um ponteiro pendente. Para essas classes é preciso definir *operador de atribuição personalizado*; tema do Capítulo 19 (*Big Three* ou, em C++ moderno, *Rule of Five*).

2 Exercício 17.2 — Caça aos erros

Encontre o(s) erro(s) em cada item e explique como corrigir.

(a) Destrutor com tipo de retorno e parâmetro

Segmento original:

```
void ~Time( int );
```

Resposta oficial: destrutores não podem retornar valores nem ter parâmetros. **Correção:** remover o tipo de retorno `void` e o parâmetro `int`.

Versão correta:

```
~Time();
```

Comentário didático

A regra é mais rígida e abrange também construtores. Em C++:

- **Construtor:** pode ter parâmetros (e sobrecargas), *não* pode ter tipo de retorno (nem `void`); seu nome é o nome da classe.
- **Destrutor:** *não* pode ter parâmetros, *não* pode ter tipo de retorno; seu nome é til (~) seguido do nome da classe; toda classe pode ter no máximo *um* destrutor (não há sobrecarga de destrutor).

Observação de Engenharia de Software

A justificativa para essas restrições é profunda: construtores e destrutores não são “chamados” como funções comuns; eles são *invocados implicitamente* pelo compilador em momentos específicos (criação e destruição do objeto). O compilador não tem nenhum lugar onde colocar o valor de retorno do destrutor — a quem ele entregaria? Nem por que escolheria um parâmetro específico no momento de destruir um objeto. Daí a proibição.

(b) Inicialização explícita na definição da classe

Segmento original:

```
class Time
{
public:
    // prototipos de funcao
private:
    int hour = 0;
    int minute = 0;
    int second = 0;
};
```

Resposta oficial: os membros não podem ser inicializados explicitamente na definição da classe. **Correção:** remover a inicialização explícita; inicializar os dados-membro em um construtor.

Versão correta:

```
class Time {
public:
    Time();           // construtor que zera tudo
private:
    int hour;
    int minute;
    int second;
};

Time::Time() : hour(0), minute(0), second(0) {}
```

Comentário importante sobre versões do C++

Observação de Engenharia de Software

Nota histórica importante: a resposta oficial reflete o padrão C++ até a versão C++03 (e o livro foi escrito em 2011). A partir do **C++11**, a linguagem incorporou um recurso chamado *default member initializer* (inicializador de membro padrão) que permite, sim, escrever `int hour = 0;` dentro da definição da classe! Em C++11 e posteriores, o segmento original *é válido*. Mas, no contexto pedagógico do livro (que adota C++03 nas primeiras edições), a resposta oficial estava correta. Adote o seguinte: *na versão didática do livro, inicialize em construtor*.

(c) Construtor com tipo de retorno

Segmento original:

```
int Employee( string, string );
```

Resposta oficial: construtores não podem retornar valores. **Correção:** remover o tipo de retorno `int`.

Versão correta:

```
Employee( string, string );
```

Por que essa regra?

Imagine que construções pudessem retornar valor:

```
Employee e("Maria", "Silva");    // o que faria com o "valor de retorno"?
```

A expressão `Employee e("Maria", "Silva")` é a inicialização de `e` — ela não é uma expressão que produza um valor consumível separadamente. O compilador não teria onde colocar um eventual valor retornado.

Erro Comum de Programação

Não confunda “construtor retorna o objeto” com “construtor tem tipo de retorno”. Construtores efetivamente *produzem* um objeto novo — mas esse objeto é o que está sendo declarado, não um valor de retorno separado. Em C++ moderno, funções *de fábrica* (*factory functions*) podem retornar objetos por valor; mas elas não são construtores — são funções comuns que internamente chamam um construtor.

Síntese conceitual

Os dois exercícios de autorrevisão do Capítulo 17 testam, em conjunto, quatro pontos essenciais:

- **Acesso a membros:** `.` para objeto/referência, `->` para ponteiro.
- **Visibilidade:** `private` encapsula, `public` expõe a interface.
- **Atribuição default:** copia membros um a um; suficiente para tipos simples, perigosa para gerenciamento de recursos.
- **Regras sintáticas de construtor e destrutor:** sem tipo de retorno; destrutor sem parâmetros e único.

Esses quatro pontos são a base sobre a qual o capítulo constrói as técnicas mais avançadas: separação interface/implementação em arquivos distintos, construtores com listas de inicialização, validação centralizada e classes que mantêm *estado coerente* mesmo diante de mudanças adversas. Aplicaremos tudo isso nos exercícios de programação 17.4 a 17.15.
